

Challenge Aqua.Bot 2024

MISSION WINDWATCH

MANUEL UTILISATEUR

I. Installation de l'environnement et prise en main



GITHUB : Afin d'installer l'environnement de simulation, veuillez suivre les étapes d'installation du fichier « `Readme` » de la compétition Aquabot au lien ci-dessous :

<https://github.com/sirehna/Aquabot#installation>

Pour installer l'exemple du package competitor, vous pouvez vous référer aux étapes d'installation du fichier `Readme Aquabot competitor` :

<https://github.com/sirehna/Aquabot-Competitor#installation>

Une fois l'environnement de développement du projet correctement installé, vous aurez récupéré et vous serez en mesure de compiler les trois dépôts de codes suivants :

- VRX (dépôt de la compétition originale)
- Aquabot (dépôt avec les adaptations pour la compétition Aquabot)
- Aquabot_competitor (dépôt avec exemples et participation des étudiants)

Si vous n'êtes pas familier avec ROS2, il est conseillé de suivre les tutoriels ROS directement sur le site ROS.

Principes fondamentaux : <https://docs.ros.org/en/humble/Concepts/Basic.html>

Tutoriels : <https://docs.ros.org/en/humble/How-To-Guides.html>

II. Lancement de l'environnement

Une fois que l'environnement est bien installé, il faudra à **chaque fois** taper les 2 commandes suivantes afin de pouvoir utiliser les commandes ROS2, et exécuter les commandes liées au colcon workspace :

```
source /opt/ros/humble/setup.bash
source ~/vrx_ws/install/setup.bash
```

Il est aussi possible de les **lancer automatiquement** à l'ouverture d'un nouveau terminal en les ajoutant au fichier "~/.bashrc". Voir le « Readme » du projet aquabot_comptetitor, section : "Permanent sourcing and aliases", qui permet aussi d'ajouter des alias permettant de faciliter le développement.

Add these lines have everythink sourced when starting new terminal :

```
# Source ROS Humble
source /opt/ros/humble/setup.bash
# Source VRX Workspace
source ~/vrx_ws/install/setup.bash
```

Add these lines to add 3 alias to build, source and code for the project :

```
# Add Alias
alias aquabot_build='cd ~/vrx_ws && colcon build --merge-install'
alias aquabot_source='source ~/vrx_ws/install/setup.bash'
alias aquabot_code='code ~/vrx_ws/src/aquabot_competitor'
```

Pour lancer un environnement de simulation il est possible de lancer le fichier « launch » `competition.launch.py` du package `aquabot_gz`.

```
ros2 launch aquabot_gz competition.launch.py world:=aquabot_regatta
```

ARGUMENTS

Il est possible d'ajouter des arguments, par exemple l'argument "world" ci-dessus permet de définir le nom de l'environnement à utiliser.

→ Un argument permet le lancement en mode compétition ("competition_mode"), un autre permet le lancement de la simulation sans l'environnement graphique ("headless"), ce qui permet d'alléger la simulation pour les PC qui ne respecteraient pas, ou aurait des difficultés par rapport à la configuration minimum requise.

Lancement d'une tâche, en compétition et sans l'environnement graphique :

```
ros2 launch aquabot_gz competition.launch.py  
world:=aquabot_windturbines_easy headless:=true  
competition_mode:=true
```

Il est possible de déplacer le bateau à l'aide d'un noeud appelé teleop_keyboard.py :

```
ros2 run aquabot_python teleop_keyboard.py
```

Un mode de pilotage avancé permet de commander les 2 moteurs séparément :

```
ros2 run aquabot_python teleop_keyboard.py -ros-args -p  
advanced:=True
```

Deux fichiers logs sont disponibles afin d'analyser le déroulement de la tâche, ils sont stockés dans -/vrx_ws/log

1. Visualisation des entrées / sorties (topics)

Pour visualiser les entrées sorties il y a **plusieurs solutions** possibles.

- a) Soit en ligne de commande, il est possible d'afficher un certain nombre de choses avec la commande "ros2 topic"

Liste des topics : ros2 topic list

Affichage du contenu d'un topic ros2 topic echo /vrx/task/info

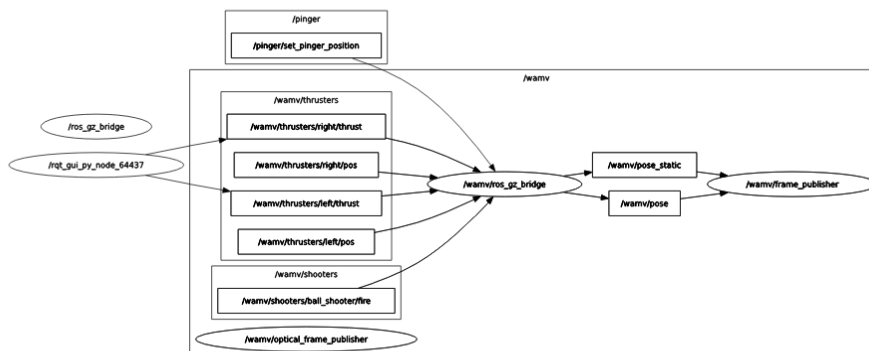
- b) Soit avec l'utilitaire rqt, en tapant la commande rqt (voir ci-dessous)

En utilisant la barre d'outils vous découvrirez de nombreux outils permettant l'affichage des topics, d'un schéma de l'environnement ros2, les logs, des graphiques ou des images. Il permet aussi de publier topics.

Voici un aperçu de la visualisation de ces topics :

☐ /vrx/debug/wind/speed	std_msgs/msg/Float32			not monitored
☒ /vrx/stationkeeping/goal	geometry_msgs/msg/PoseStamped	unknown	0.94	
header	std_msgs/Header			
stamp	builtin_interfaces/Time			
frame_id	string			"
pose	geometry_msgs/Pose			
position	geometry_msgs/Point			
x	double			-33.722718
y	double			150.674031
z	double			0.0
orientation	geometry_msgs/Quaternion			
x	double			0.0
y	double			0.0
z	double			0.0
w	double			1.0
☒ /vrx/stationkeeping/mean_pose_error	std_msgs/msg/Float32	unknown	0.94	
data	Float			2.4972426891326904
☒ /vrx/stationkeeping/pose_error	std_msgs/msg/Float32	unknown	0.94	
data	Float			1.2801271677017212

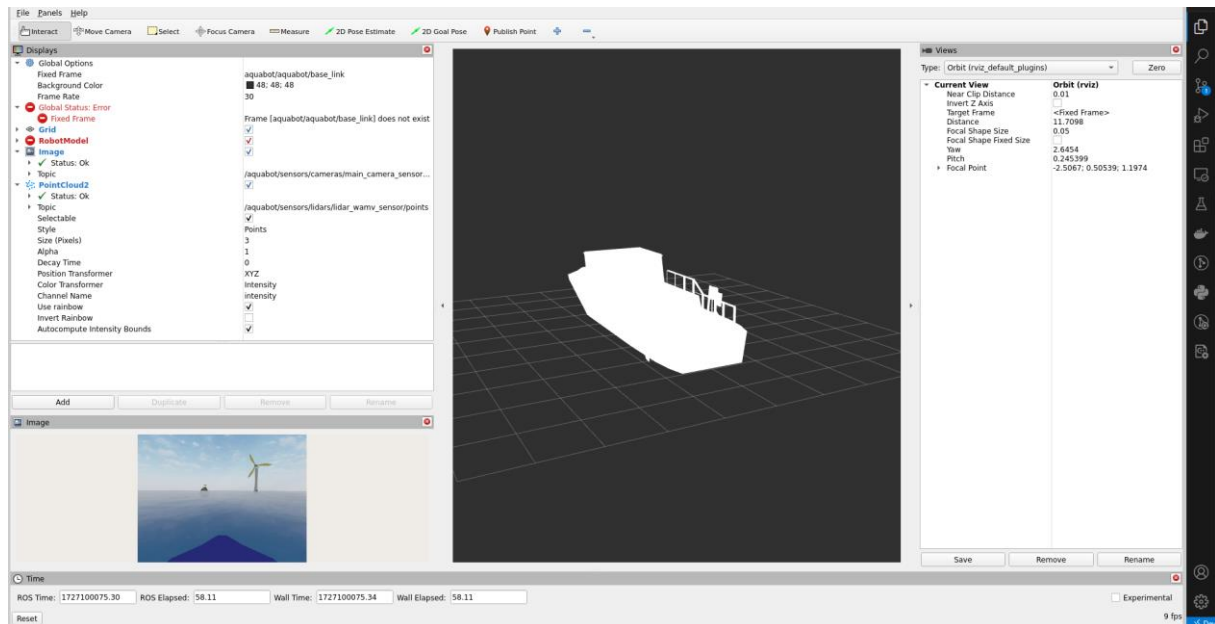
Voici un aperçu de l'affichage d'un schéma des nœuds et topics en cours d'exécution. (Outil Node Graph)



La dernière manière de visualiser l'environnement ROS2 est l'outil **rviz2**.

Cet outil permet la **visualisation graphique en 3D** des informations à votre disposition. Dans notre cas, cet outil sera un peu plus compliqué à prendre en main car nécessite la connaissance et l'utilisation des "TF2 : TransFormations ROS2" permettant de positionner le bateau par rapport à son environnement dans un repère cartésien.

Voici un aperçu :



Sur cette image on peut voir l’affichage de la caméra en bas à gauche et du lidar sur l’affichage 3D (**lidar non disponible durant la compétition 2024**). Il est aussi possible d’ajouter des topics permettant l’affichage d’autres informations sur rviz (tel que des marqueurs) qui vous permettront de mieux comprendre les développements que vous réaliserez.

Ces outils sont suffisamment pratiques pour pouvoir se passer de l’environnement graphique Gazebo. Il est possible d’enregistrer vos propres configurations de rviz2.

III. Développement de votre solution

Votre programme devra être réalisé sous forme d’un ou plusieurs nœuds ROS afin de pouvoir lire et écrire sur les topics. Les nœuds ROS sont des exécutables pouvant interagir entre eux via l’utilisation des topics et des services.

Vous aurez le choix entre deux langages de programmation, le C++ et le Python. Vous pouvez utiliser l’éditeur de code qui vous convient.

Il y a 3 packages d’exemple dans le dépôt aquabot competitor,

- a) package_example : Contient des exemples ros2 en python et en C++
- b) opencv_example : Contient des exemples d’utilisation de la librairie OpenCV
- c) aquabot_example : Contient des exemples liés à l’environnement aquabot

Dans le package `aquabot_exemple`, vous trouverez également un exemple de fichier `launch` permettant le lancement d'un nœud en même temps que la simulation `aquabot`. (Permet de tout lancer en une seule commande).

Afin de développer une solution, vous avez le choix de créer votre propre dépôt (de la même façon que le dépôt `aquabot_competitor`). Ou alors d'utiliser le dépôt `aquabot_competitor` en réalisant **un fork** pour votre développement.

Vous devrez fournir à la restitution de votre dossier, votre dépôt accompagné d'une commande permettant de lancer votre solution.

Il est autorisé d'installer des packages supplémentaires, de la même façon que `OpenCV`. Il est demandé de n'utiliser que des packages installables avec une commande `apt install` et de signaler leur(s) ajout(s) aux organisateurs.

ENTRAINEMENTS

Un script de génération de 10 scénarios aléatoires est disponible dans le dépôt `aquabot`. Voici les instructions pour l'utiliser :

```
cd ~/vrx_gz/aquabot/aquabot_python/scripts
python3 generate_competition_worlds.py
```

Les scénarios sont plus ou moins difficile en fonction de la proximité et la quantité des éoliennes, le positionnement des obstacles fixes (rochers).

IV. Capteurs de navigation

La simulation `Aquabot` permet de simuler le comportement simplifié d'un bateau dans un environnement maritime. Le bateau possède plusieurs capteurs et actionneurs.

Les capteurs correspondent à :

- Une caméra couleur 360°
- Une IMU (unité de mesure inertielle)
- Un GPS
- Un capteur acoustique.

Le bateau dispose de deux actionneurs :

- Deux moteurs à hélices orientables

Un seul capteur de navigation standard, représentant une centrale inertielle (IMU) assistée par GPS, sera utilisé pour le challenge. Ce capteur unique est simulé à l'aide des capteurs NavSatSensor et ImuSensor de Gazebo (voir gz-sensors), qui génèrent des informations GPS (position et vitesse) et des informations IMU (attitude, taux d'attitude et accélérations). Bien que ces deux mesures soient présentées séparément, les caractéristiques des mesures sont cohérentes avec un capteur qui estime une solution de navigation complète, par exemple, une centrale inertielle assistée par GPS.

GPS	
Update Rate	20 Hz
Horizontal Position Noise1	0.85 m
Vertical Position Noise	2.0 m
Velocity Noise	0.1 m/s

IMU	
Update Rate	100 Hz
Acceleration Offset/Bias	+/- 0.002 g
Acceleration Noise	0.275 g
Attitude Rate Noise	0.08 degrees/s
Heading Noise	0.8 degrees

Camera 360°	
Update Rate	15 Hz
Résolution	1280x720 px
Color format	R8G8B8

V. Système de propulsion

Le bateau virtuel peut-être manœuvré à l'aide de 2 moteurs.

Le système de propulsion est contrôlable via 2 topics par moteur :

- **Position cible des moteurs :**

`/aquabot/thrusters/left/pos` et `/aquabot/thrusters/right/pos`

Correspondant à la position du moteur à l'arrière du bateau (angle en radian), valeur min $-\pi/4$, valeur max $\pi/4$.

- **Vitesse de l'hélice :**

`/aquabot/thrusters/left/thrust` et `/aquabot/thrusters/right/thrust`

Correspondant à la vitesse de rotation de l'hélice, min -5000 , max 5000 .

Il est possible de récupérer la position réelle des moteurs en utilisant les transform (tf2) ROS. Un exemple est fourni dans le package `aquabot_example`.

La vitesse maximale du bateau joueur est de 12 nœuds.

VI. Pilotage de la caméra

Dans cette nouvelle édition 2024, la caméra est **pilotable à 360 degrés**. Pour l'orienter un nouveau topic `/aquabot/thrusters/main_camera_sensor/pos` permet de piloter son moteur en lui fournissant l'angle en radians.

La rotation n'est pas instantanée, il faudra attendre quelques instants avant que la caméra n'arrive à sa position. (Déplacement de 180 degrés en moins d'une seconde).

Astuce : Pour aller d'un angle de -150 degrés à 150 degrés, il vaudra mieux envoyer -210 . L'angle parcouru par la caméra dépend de la différence avec l'angle cible précédemment envoyé.

VII. API Aqua.Bot

L'interface ROS 2 permet de contrôler tous les actionneurs disponibles, lire les informations des capteurs et envoyer/recevoir des notifications.

Les tableaux suivants résument l'API ROS2 utilisée pour la compétition :

Thruster actuation API :

Thruster Actuation	
Topic Name	Description
/aquabot/thrusters/left/pos	Next angle command for the left thruster
/aquabot/thrusters/left/thrust	Next power command for the left thruster
/aquabot/thrusters/right/pos	Next angle command for the right thruster
/aquabot/thrusters/right/thrust	Next power command for the right thruster
/aquabot/thrusters/main_camera_sensor/pos	Next angle command for the camera

Sensor information API :

Sensor Information	
Topic Name	Description
/aquabot/sensors/cameras/main_camera_sensor/camera_info	Meta information for camera. See CameraInfo message for details.
/aquabot/sensors/cameras/main_camera_sensor/image_raw	Raw image data for camera. See Image message for details.
/aquabot/sensors/gps/gps/fix	GPS position data.
/aquabot/sensors/imu/imu/data	IMU data describing orientation, angular velocity and linear acceleration.
/aquabot/sensors/acoustics/receiver/range_bearing	A distance (range) and two angles (bearing and elevation) indicating the relative position of the acoustic pinger from the USV, with noise.
/aquabot/ais_sensor/windturbines_positions	Pose list GPS of the windturbines (0.1 Hz)

Alert API :

Alert Information	
Topic Name	Description
/vrx/windturbineinspection/windturbine_checkup	Topic to send the reports of the windturbines markers during the search phase (first phase)

Task information API :

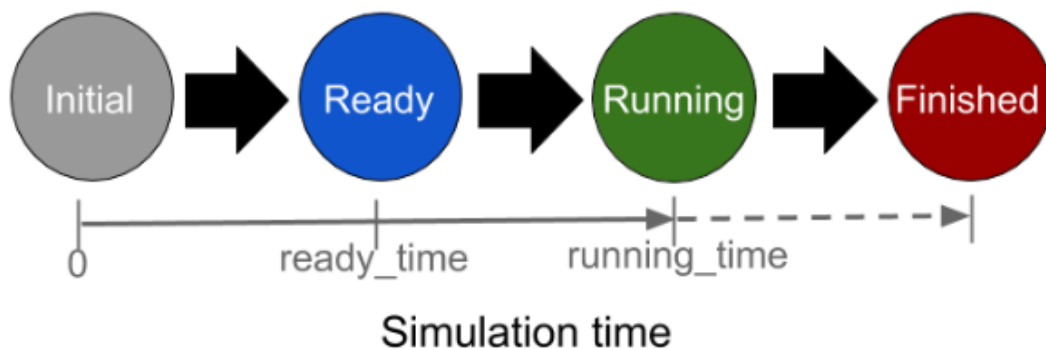
Tasks	
Topic Name	Description
/clock	Simulation time
/vrx/task/info	Task information
/vrx/windturbineinspection/current_phase	Current phase of the task, see information about task below

Debug information API (not available during competition) :

Debug	
Topic Name	Description
/vrx/debug/wind/direction	Wind vector (units in degrees and ENU coordinate)
/vrx/debug/wind/speed	Magnitude of the wind
/vrx/windturbineinspection/debug/critical_windturbine_heading	Bearing between the player and the critical windturbine (Ex: 0 = the player boat is facing the critical windturbine)
/vrx/windturbineinspection/debug/critical_windturbine_distance	Distance between the player and the critical windturbine
/vrx/windturbineinspection/debug/critical_windturbine_bearing_from_boat	Vector direction between the player and the critical windturbine

/vrx/windturbineinspection/debug/critical_target_pose_distance	Target pose distance, the target pose is 10 meters in front of the windturbine marker
/vrx/windturbineinspection/debug/critical_turn_around_angle	Angle traveled around the windturbine since the turn around phase
/vrx/windturbineinspection/debug/windturbines_distances	Windturbines distances
/vrx/windturbineinspection/debug/windturbine_check_status_ok	Number of windturbines found with the correct state
/vrx/windturbineinspection/debug/windturbine_check_status_not_received	Number of windturbines not found

VIII. Structure d'une tâche



Les tâches peuvent se trouver dans l'un des quatre états suivants :

- **L'état initial « Initial »**, où le robot est limité en mouvement pour permettre la dissipation des transitoires de démarrage ;
- **L'état prêt « ready »**, où le robot a une mobilité totale sous le contrôle du participant sans score en cours ;
- **L'état en cours « running »**, qui démarre officiellement la tâche avec score et chronomètre activés ;
- **L'état terminé « finished »**, atteint lorsque le temps restant atteint zéro ou lorsque la tâche est considérée comme complète.

Dans notre cas, le temps à l'état initial est de 10 secondes, le temps à l'état ready est de 10 secondes et le temps à l'état running est de 1200 secondes (20 minutes).

Message task info :

Voici le contenu du message /vrx/task/info

Parameter Name	Description
name	Unique task name (e.g.: "windturbines_inspection").
state	The current task state = {initial, ready, running, finished}. See the Task States section for more information.
ready_time	Simulation time at which the task transitions to the ready state.
running_time	Simulation time at which the task transitions to the running state.
elapsed_time	Elapsed time since the start of the task (since running_time).
remaining_time	Remaining task time.
timed_out	Whether the task has timed out or not.
score	Current task score.
num_collisions	Number of times the vehicle has collided with a course element.

IX. Tâches à effectuer

L'automatisation complète de votre USV est demandée.

Les modules suivants devront être développés :

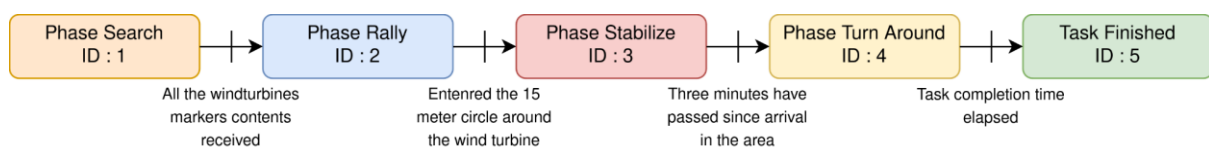
- Un module de perception pour traiter les données capteur, détecter les éoliennes et lire les QR code.
- Un module de guidage et contrôle de l'USV faisant les choix de route et envoyant les ordres de contrôle à la propulsion.

Dans le cadre de l'opération de maintenance, le drone doit réaliser 3 tâches successives obligatoire + 1 tâche BONUS

L'environnement maritime contient de nombreux obstacles fixes, la position du phare et des rochers est statique entre les scénarios, la position et l'état des éoliennes sera généré aléatoirement pour chaque scénario.

Le bateau joueur démarrera toujours de la même position. Une tâche facile avec un environnement éloigné a été créé pour débiter dans un environnement simplifié mais ne sera pas utilisé pour la compétition.

La tâche doit être réalisée en plusieurs phases.



INSPECTION

du statut de chaque éolienne

IDENTIFICATION

de l'éolienne défaillante dans le champ

STABILISATION & MAINTENANCE

devant l'éolien défaillante

ROUND TRIP

Effectuer 2 tour complet de l'éolienne selon un point fixe

1. ETAPE 1 INSPECTION : *(Temps recommandé : 10 min)*

Votre USV doit naviguer au sein du champ éolien dans le but de visiter et contrôler l'état des éoliennes en renvoyant leur statut (« fonctionnel » ou « défectueux ») grâce aux QR codes positionnés au bas de chacune, et orienté aléatoirement à chaque scénario. Avant l'initialisation du scénario les positions des éoliennes ne sont pas connues, elles sont transmises aux compétiteurs toutes les 10s à partir du début du scénario.

Lors de la première phase (Search), l'objectif est de remonter l'état de toutes les éoliennes, en décodant et en envoyant le contenu de leur QR code. La position GPS des éoliennes est envoyée le topic /aquabot/ais_sensor/windturbines_positions.

2. ETAPE 2 IDENTIFICATION : *(Temps recommandé : 2 min)*

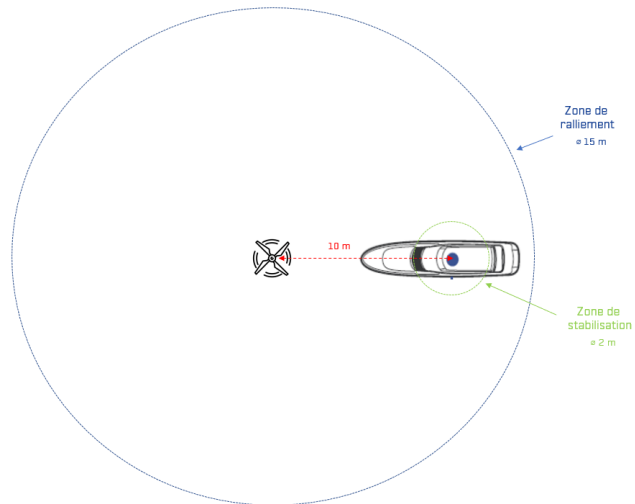
- Si toutes les positions des éoliennes ont bien été trouvées et renvoyées durant l'étape 1 → un PING vous sera envoyé pour indiquer l'ID de l'éolienne à visiter avec un état « critique »
- Si l'USV n'a pas visité ou renvoyé les ID de toutes les éoliennes durant l'étape 1 → au bout d'un certain temps, l'ID de l'éolienne à visiter avec un état « critique » vous sera automatiquement envoyée avec le déclenchement de l'AIS qui émettra la position de celle-ci.

3. Etape 3 STABILISATION & MAINTENANCE : *(Temps recommandé : 3 min)*

Avec l'ID de l'éolienne désignée comme « critique », votre USV doit aller se positionner puis stabiliser sa position en face de l'éolienne à 10m de distance entre l'axe de l'éolienne et l'axe du navire, en restant dans un cercle de 2m de diamètre, la proue faisant face au QR code de cette éolienne pendant 30 seconde.

Une fois que l'USV aura pénétré une zone de 15 mètres de diamètre autour de l'éolienne, un chrono de 3 minutes s'enclenchera. Vous aurez 3 minutes pour compléter l'étape 3.

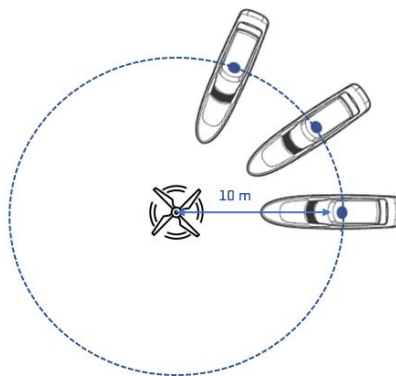




4. Etape 3 BONUS « ROUND TRIP » :

Votre USV doit faire deux tours complet (360°) de l'éolienne avec maintien de la position et de la distance (10m) de la proue du navire vers l'éolienne.

La dernière phase est une phase bonus dont l'objectif est de garder la proue du bateau alignée sur l'éolienne critique et de faire des tours en maintenant une distance de 10 mètres.



Les éléments d'un scénario pouvant être amenés à changer sont :

- La position, l'orientation et l'état des éoliennes.
- Le vent et les vagues peuvent aussi être modifiés. Nous fixerons des valeurs maximales pour le vent et les vagues au début de la compétition. Il est aussi possible d'ajouter de la lumière ambiante et du brouillard, mais ceux-ci ne seront pas modifiés.

X. Notation

Chaque équipe devra fournir avant la date spécifiée :

1. Le **code source** des modules développés compatibles avec l'environnement de simulation fournit.
2. **Un rapport** précisant :
 - L'organisation de l'équipe mise en place dans le cas d'un développement collectif.
 - Une description des algorithmes mis en place.
 - Une notice d'installation (préciser le cas échéant la méthode d'installation de librairies complémentaires par exemple).
 - Les difficultés rencontrées et leur résolution si résolus.

→ Il sera également demandé à l'issue de la phase d'IDEATION une présentation des équipes et projets.

Chaque équipe recevra une note qui sera la somme d'une **évaluation sur le rapport**, d'une **évaluation sur la performance** et d'une **évaluation sur leur pitch** de présentation durant la finale.

Le nombre de point total est **de 75 points**, répartis ainsi :

- 15 pour points pour l'évaluation du rapport et du code
- 40 points pour l'évaluation de performance

Soit un total de **55 points** avant les pré-sélections, puis :

- 20 points lors de la présentation finale devant jury

1 . Evaluation du rapport et de la qualité du code: (15 points)

- Qualité générale du rapport : observation et design thinking, gestion de projet et roadmap.
- Niveau technique des solutions proposées, et clarté de la description.

- Qualité générale du code livré.
- 15 pages maximum.

Une pénalité par jour de retard (date de réception, quelle que soit l'heure) sur la livraison du code source et/ou du rapport sera comptabilisée.

2.Evaluation de la performance (40 points)

La notation principale de la compétition se fait selon un certain nombre de critères pour chaque phase.

La note sera moyennée sur N scénarios (autres que ceux transmis pour l'entraînement).

Les critères de notations sont :

- La bonne remonté de l'état des éoliennes
- Le temps de réalisation de la phase 1.
- Le temps de ralliement lors de la phase 2.
- La précision du maintien de la position face à l'éolienne critique (moyenne) (phase 3)
- La précision du maintien du cap face à l'éolienne critique (moyenne) (phase 3)
- La précision du maintien de la distance de 10 mètres lors des tours (phase 4)
- La précision du maintien du cap face à l'éolienne critique (moyenne) (phase 4)
- L'évitement des obstacles (fin du scénario en cas de choc)

Un classement des équipes sera effectué à l'issue de cette phase de notation sur 55 points.

Le classement global de la compétition dépendra du classement de votre équipe pour chacun des critères de notation ci-dessus.

Les scénarios utilisés pour la notation finale seront générés aléatoirement. Pour l'entraînement, 3 exemples de scénarios sont disponibles avec un niveau de difficulté progressif.

Toute collision entrainera la fin du scénario. Les points accumulés avant la collision resteront comptabilisés

Seront comptabilisés comme collision : La collision avec un obstacle fixe (rochers, phare, éolienne...).

Une inspection du code livré sera réalisée pour identifier toute tentative de triche visant à extraire de l'environnement de développement la position réelle des bateaux alliés ou menace. En cas de doute sur ce qui autorisé ou non, contacter l'équipe organisatrice.

Evaluation du pitch: (20 points)

Une finale réunissant la 10 meilleure équipe présélectionnée sera organisée dans les locaux de Sirehna, au Technocampus Océan, 5 rue de l'Halbrane, 44340 BOUGUENAI, France.

Durant la finale, votre équipe devra présenter ses travaux pendant un pitch de courte durée (entre 5 et 7 minutes) devant un jury, à l'aide d'un support visuel composé d'un maximum de 10 slides pour votre pitch-deck. A l'issue des présentations, une note vous sera attribuée.

Celle-ci sera alors pondérée avec votre rapport et la performance de votre USV durant les tests.

Les équipes victorieuses seront annoncées durant l'événement final.

Les trois (3) équipes gagnantes seront déterminées par l'addition de l'ensemble des notations reçues. Des prix spéciaux pourront également être accordés à certaines équipes, même des équipes n'ayant pas été présélectionnées (Meilleure participation, meilleure présentation etc.).

ⁱ Composition « type » du pitch-deck : page de garde, présentation du challenge, présentation de la compréhension du contexte, présentation de votre équipe projet et de ses compétences, présentation de l'organisation et de la gestion de votre projet, présentation de l'idée, des choix et du travail réalisé en 2-3 slides, présentation des résultats, mise en lumière des points d'amélioration potentiels dans le cadre où votre projet n'est pas finalisé et/ou si vous possédez des idées de features à développer par la suite, vos contacts.